

Building a Bank's Governed MCP Tool Platform

The enterprise problem of *scale*: dozens of systems, many teams building agents. Solve it with a reusable, secured MCP layer — servers, AgentCore Gateway, governance — shared by production agents and developer assistants alike.

Prepared for Kaniz

Domain: financial services / retail banking.

Builds on Guide 1 (Foundations) and Guide 2 (Enterprise Agentic).

How to read this guide

Each section: the concept, then the build steps, then real code. Blue = key idea, green = best practice, orange = pitfall, dark = runnable code / CLI. Adapt URLs, ARNs and schemas to your estate.

Contents

- 1 The problem: tool sprawl at enterprise scale
- 2 Platform architecture (MCP + Gateway + governance)
- 3 Phase 0 — Foundations & account setup
- 4 Phase 1 — Design an MCP server (tools, resources, prompts)
- 5 Phase 2 — Build the Customer-360 MCP server (FastMCP)
- 6 Phase 3 — Transports: stdio vs streamable HTTP
- 7 Phase 4 — Deploy the MCP server to AgentCore Runtime
- 8 Phase 5 — Wrap existing APIs via Gateway (build vs wrap)
- 9 Phase 6 — Aggregate servers behind one Gateway
- 10 Phase 7 — Security: ingress, egress & Policy
- 11 Phase 8 — Connect consumers (Claude + agents)
- 12 Phase 9 — Governance, versioning & a tool catalog
- 13 Phase 10 — Testing & observability for the MCP layer
- 14 Checklists, failure modes & pitfalls

1 · The problem: tool sprawl at enterprise scale

The bank has dozens of systems: core banking, cards, fraud, CRM, KYC, payments, statements, the knowledge base. Multiple teams are now building agents — the disputes agent from Guide 2, a relationship-manager copilot, a collections assistant, an internal analyst tool. Without a standard, **every team re-integrates every system**: duplicated glue code, inconsistent auth, no central audit, and a security review per integration. That's the **N×M explosion** — M agents × N systems.

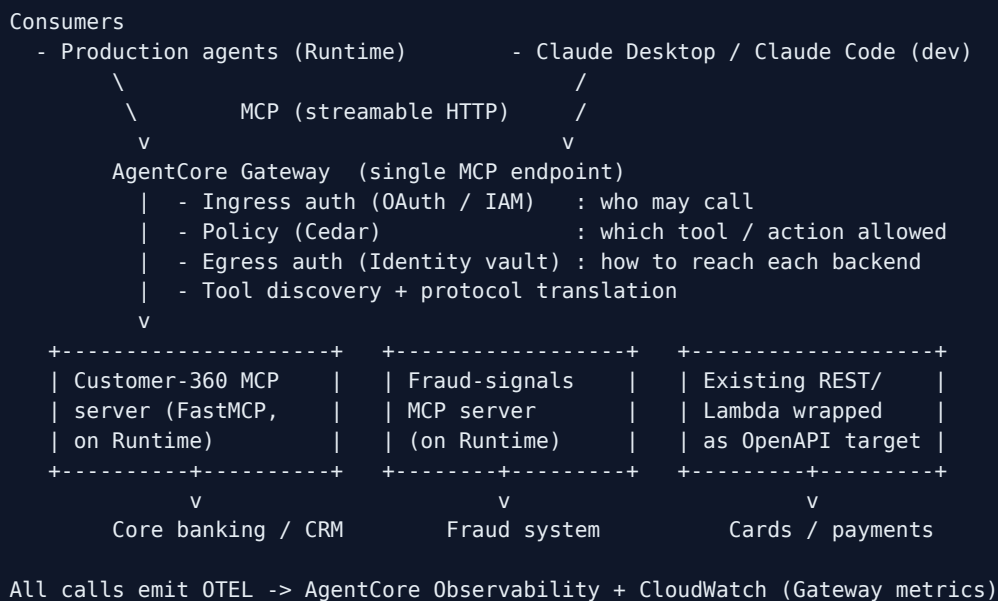
The platform thesis

Wrap each system **once** as a governed MCP capability. Then any agent — and any developer's coding assistant — consumes it with zero bespoke integration, one auth model, and one audit trail. N×M becomes N+M. This is the enterprise payoff of MCP, and it's a platform, not a feature.

What we're building

A **governed MCP tool platform**: a small set of MCP servers exposing bank capabilities (starting with a **Customer-360** server), fronted by **AgentCore Gateway** for auth, aggregation and policy, with a catalog, versioning, testing and observability — consumed by production agents and by Claude in developers' IDEs.

2 · Platform architecture



Two ways a capability reaches the platform, and you'll use both:

Approach	Use when	Mechanism
Build an MCP server	You need logic/aggregation beyond the raw API (e.g. Customer-360 joins 3 systems)	FastMCP server on Runtime
Wrap an existing API	A clean API/Lambda already exists	Gateway OpenAPI / Lambda / Smithy target

3 · Phase 0 — Foundations & account setup

Same landing-zone baseline as Guide 2 (orgs, SSO, CloudTrail, KMS, budgets, IaC). Two MCP-specific additions:

- A **platform account/team** that owns the MCP servers and Gateway, separate from consuming teams — the platform is a shared service with its own ownership and on-call.
- Enable Bedrock model access (consumers need it) and pick a Region with AgentCore Runtime + Gateway.

Treat the MCP layer as a product

It has consumers, SLAs, versioning and a support model. Decide ownership now; an ungoverned pile of MCP servers is just distributed tool sprawl with extra steps.

4 · Phase 1 — Design an MCP server

An MCP server can expose three kinds of things. Know the difference — most enterprise value is in tools, but resources and prompts matter:

Primitive	What it is	Banking example
Tool	A callable action the model can invoke	get_customer_360, list_transactions
Resource	Readable data the host can pull into context	A customer's redacted profile document
Prompt	A reusable, parameterised prompt template	'triage_dispute' canned workflow prompt

Design rules (from Guide 1, applied)

- One server = one bounded capability (Customer-360, Fraud, Payments) — not one giant server.
- Tight Pydantic/typed schemas; clear docstrings (they become the tool descriptions the model reads).
- Read vs write separated; writes idempotent; outputs small + structured + PII-minimised.
- Version the contract; never silently change a tool's shape (consumers depend on it).

5 · Phase 2 — Build the Customer-360 MCP server (FastMCP)

Customer-360 is a good 'build' case: it *joins* core banking, CRM and KYC into one clean view — logic no single API provides. We build it with FastMCP, configured stateless for Runtime.

```
# customer360_server.py
from mcp.server.fastmcp import FastMCP
from pydantic import BaseModel, Field

# stateless_http=True is required for AgentCore Runtime hosting
mcp = FastMCP('customer360', stateless_http=True, host='0.0.0.0', port=8000)

class Customer360(BaseModel):
    customer_id: str
    name: str
    verification_status: str = Field(description='verified | pending | failed')
    products: list[str]
    risk_tier: str

@mcp.tool()
def get_customer_360(customer_id: str) -> Customer360:
    """Unified customer view joining core banking, CRM and KYC. Read-only."""
    core = core_bank.profile(customer_id)
    crm = crm_api.summary(customer_id)
    kyc = kyc_api.status(customer_id)
    return Customer360(customer_id=customer_id, name=core['name'],
        verification_status=kyc['status'], products=core['products'],
        risk_tier=crm['risk_tier'])

@mcp.tool()
def list_transactions(customer_id: str, days: int = 30) -> list[dict]:
    """Recent transactions for a customer. Read-only."""
    return core_bank.transactions(customer_id, days)

if __name__ == '__main__':
    mcp.run(transport='streamable-http') # serves /mcp on :8000
```

Pitfall: stateful by default
Default MCP servers keep per-connection state, which breaks in a scaled, session-isolated Runtime. Set `stateless_http=True` and don't rely on in-memory state between calls — persist anything durable to DynamoDB.

6 · Phase 3 — Transports: stdio vs streamable HTTP

Transport	Where the server runs	Use for
stdio	Local subprocess of the host	Dev, Claude Desktop/IDE on your laptop, quick tools
Streamable HTTP	A remote endpoint	Production, shared/multi-consumer servers, Runtime hosting

The same FastMCP code supports both — you only change the transport. Develop locally over stdio, then deploy the identical server over streamable HTTP to Runtime. AgentCore expects the MCP server at `0.0.0.0:8000/mcp` and adds an `Mcp-Session-Id` header so clients

keep continuity to the same Runtime session.

7 · Phase 4 — Deploy the MCP server to AgentCore Runtime

Hosting your MCP server on Runtime gives it the same serverless scaling, session isolation, identity and observability your agents get. Deploy with the toolkit, declaring the **MCP protocol**.

```
# requirements.txt -> mcp, pydantic, (your backend SDKs)
# Deploy with protocol = MCP
agentcore configure -e customer360_server.py --protocol MCP
agentcore launch      # builds container (server on :8000/mcp), pushes to ECR, creates Runtime

# The Runtime now exposes a secured MCP endpoint (HTTPS) for this server.
```

- Versions + endpoints work exactly as for agents: immutable versions, dev/staging/prod endpoints, instant rollback.
- Runtime injects identity + auth; the server itself stays simple.
- For long-running tools, Runtime supports extended execution and async patterns.

When to host on Runtime vs front with Gateway only
Host on Runtime when you've *built* a custom server with logic (Customer-360). Use Gateway alone when you're just *wrapping* an existing API (next). Most platforms do both and put Gateway in front of everything.

8 · Phase 5 — Wrap existing APIs via Gateway (build vs wrap)

Not everything deserves a hand-built server. For a clean existing API or Lambda, let **Gateway** turn it into MCP tools with zero server code — the same recipe as Guide 2, Phase 3.

- 1 Upload the API's **OpenAPI** spec to S3 (clear per-operation summaries = tool descriptions).
- 2 Create a Gateway target: **OpenAPI** (REST), **Lambda** (functions), or **Smithy** (AWS-native).
- 3 Attach **egress auth** via an Identity credential provider (OAuth/API key/IAM) — secret vaulted.
- 4 Gateway publishes each operation as an MCP tool at the Gateway endpoint.

Decision	Build a server	Wrap via Gateway
Need to join/transform data	Yes	No
Clean API already exists	Maybe	Yes

Decision	Build a server	Wrap via Gateway
Custom logic / caching / guards	Yes	Limited
Fastest time-to-tool	No	Yes

9 · Phase 6 — Aggregate servers behind one Gateway

Consumers should see **one endpoint**, not five. Gateway can aggregate multiple targets — including **existing MCP servers** as a target type — behind a single managed MCP interface. Your Customer-360 server, the Fraud server, and the wrapped REST APIs all surface as one discoverable tool set.

- One endpoint URL, one auth handshake, one place to apply Policy and read metrics.
- Add/remove capabilities without changing consumer config — they just see new tools appear in discovery.
- Namespacing keeps tool names unambiguous across servers.

This is the platform moment

A single governed Gateway endpoint + a catalog is the product you hand to every agent team. They point one MCP client at it and get the bank's approved capabilities, already secured and audited.

10 · Phase 7 — Security: ingress, egress & Policy

Ingress (who may call the platform)

- OAuth via the bank IdP (Cognito/Okta/Entra) or IAM SigV4. Tokens carry scopes per consuming team/role.
- Different consumers get different tool visibility and limits — the collections agent need not see dispute-write tools.

Egress (how the platform reaches backends)

- Backend credentials vaulted in **AgentCore Identity**; injected at call time; never in server code, logs, or memory.
- Choose **user-delegated** (act as the signed-in user; best for audit attribution) or **autonomous** (service identity) per tool.

Policy (which tool / action is allowed, deterministically)

```
// Cedar-style platform rules (illustrative)
permit (principal, action, resource)
when { principal.team in ["disputes","rm-copilot"] && action.kind == "read" };

forbid (principal, action, resource)
when { action.kind == "write" }
unless { principal.team == "disputes" && context.human_approved == true };
```

Pitfall: trusting the prompt for authz

Never rely on the system prompt to stop a consumer from calling a tool. Enforce visibility and limits at the Gateway with ingress scopes + Cedar Policy. The prompt is UX; Policy is security.

11 · Phase 8 — Connect consumers (Claude + agents)

A production agent (Strands) → the Gateway

```
from strands import Agent
from strands.models import BedrockModel
from strands.tools.mcp import MCPClient
from mcp.client.streamable_http import streamablehttp_client

gw = MCPClient(lambda: streamablehttp_client(GATEWAY_MCP_URL,
      headers={'Authorization': f'Bearer {token}'}))

with gw:
    tools = gw.list_tools_sync()          # Customer-360, fraud, wrapped APIs
    agent = Agent(model=BedrockModel(model_id=MODEL_ID), tools=tools,
                  system_prompt=SYSTEM)
    print(agent('Give me a 360 view + recent disputes for customer 4471').message)
```

Claude Desktop / Claude Code (developer productivity)

Developers can point their assistant at the *same* governed platform — so the tools they build agents with and the tools they explore interactively are identical and equally audited. Local stdio servers go straight into the MCP config; a remote, auth-protected Gateway endpoint is reached via a custom/remote MCP connector (e.g. an *mcp-remote* style bridge that handles the OAuth handshake).

```
// Claude Desktop MCP config – local stdio server during development
{
  "mcpServers": {
    "customer360": { "command": "python", "args": ["/abs/customer360_server.py"] }
  }
}
```


The MCP + AgentCore + Claude loop

Because the platform speaks MCP throughout, Claude in the IDE can help *build* new servers/tools, the Gateway *governs* them, agents in Runtime *consume* them, and Observability *audits* every call — one protocol, end to end. (AWS even publishes an AgentCore MCP server that gives the assistant first-class knowledge of AgentCore APIs.)

12 · Phase 9 — Governance, versioning & a tool catalog

- **Tool catalog/registry** — a discoverable list of every tool: owner, schema, data class, auth required, status (active/deprecated). This is how teams find approved tools instead of reinventing them.
- **Versioning** — treat tool contracts like APIs: additive changes are safe; breaking changes get a new version and a deprecation window. Runtime versions/endpoints give you the deploy mechanics.
- **Onboarding** — a lightweight intake (schema review, data-class + PII check, security sign-off) before a new tool joins the platform.
- **Ownership + SLA** — each server has an owning team, on-call, and a support channel.

Make adding a tool boring

A documented intake + IaC template + automated tests means a new bank capability becomes a platform tool in days, safely. That repeatability is the whole point of building a platform rather than one-off integrations.

13 · Phase 10 — Testing & observability for the MCP layer

- **MCP Inspector** — interactively connect to a server, list tools, and call them during development to verify schemas and behaviour before any agent touches it.
- **Unit + contract tests** — test each tool function with mocked backends; assert the schema contract so a refactor can't silently break consumers.
- **Gateway observability** — metrics per target: invocations, concurrency, latency/duration, target-execution time, throttles, system/user errors, requests-per-session. Alarm on error rate, latency, and cost.
- **End-to-end traces** — OTEL spans show a consumer's call → Gateway → Policy decision → backend → result, so you can pinpoint whether a failure is the agent, the platform, or the backend.

14 · Checklists, failure modes & pitfalls

Recipe — add a new MCP tool to the platform

- 1 Decide build vs wrap; if building, scaffold a FastMCP server (stateless_http=True), one bounded capability.
- 2 Define typed schemas + clear docstrings; separate read/write; make writes idempotent; minimise PII in outputs.
- 3 Test with MCP Inspector + unit/contract tests.
- 4 Deploy (Runtime --protocol MCP, or Gateway target for a wrap); register the egress credential in Identity.
- 5 Front with the shared Gateway; add ingress scopes + Cedar Policy; add to the catalog with owner + data class.
- 6 Wire metrics/alarms; publish the version; announce to consumers.

Recipe — onboard a new consumer (agent or developer)

- 1 Issue an IdP client + scopes for exactly the tools they need.
- 2 Add Policy rules for their team/role (read vs write, approval requirements).
- 3 Hand them the single Gateway MCP endpoint URL; they point their MCP client at it.
- 4 Verify discovery shows only permitted tools; run one call; confirm it appears in traces/metrics.

Failure modes & pitfalls

Pitfall	Mitigation
One giant 'do-everything' server	One bounded capability per server; aggregate at the Gateway
Stateful server on scaled Runtime	stateless_http=True; persist to DynamoDB, not memory
Breaking a tool's schema	Version contracts; additive changes only; deprecation window
Authz via prompt	Ingress scopes + Cedar Policy at the Gateway
Secrets in server code	Vault in Identity / Secrets Manager; inject at call time
Untracked tool sprawl	Catalog + intake + ownership; platform-as-product
No layer-level tracing	Gateway metrics + OTEL end-to-end spans + alarms

You've now seen both sides

Guide 2 built one enterprise **agent**; Guide 3 built the reusable **MCP platform** the whole bank shares. Together they're how agentic systems scale safely in a regulated enterprise: governed tools once, many agents on top, every call authorized, audited and observable.

Sources: AWS Bedrock AgentCore developer guide (Runtime MCP hosting, Gateway targets & MCP aggregation, Identity, Policy, Observability/Gateway metrics); AWS ML blog on MCP servers + Strands; modelcontextprotocol.io. docs.aws.amazon.com. Verified May 2026.